

05_explainability

November 24, 2025

0.1 1. Imports and Path Setup.

Configure project directories, hardware device, and ensure internal /src modules are accessible for explainability workflows. ##### Rationale: - Establish a reproducible environment consistent with previous notebooks (01 to 04 notebooks). - Resolve canonical project paths and verify the presence of core directories (data/, experiments/, reports/). - Create an output subfolder under /reports/figures/explainability to store Grad-CAM and LIME visual artefacts. - Ensure /src modules (dataset, transforms, loaders) are discoverable by appending the project root to sys.path. - Detect available compute device (GPU or CPU) to confirm runtime environment for model interpretation.

```
[1]: # 1. Imports and Path Setup.
# Configure directories, device, and ensure /src modules are visible.
# The structure remains consistent with 01-04 notebooks.

import sys, json, numpy as np, torch
import torch.nn as nn
from torchvision import models, transforms
import matplotlib.pyplot as plt
from pathlib import Path
from PIL import Image
import pandas as pd
from sklearn.metrics import confusion_matrix

# Project structure.
PROJECT_ROOT = Path("..").resolve() # Go up one level from /notebooks
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

print(f'Project root added to path: {PROJECT_ROOT}')

DATA_DIR = PROJECT_ROOT / "data"
EXPERIMENTS = PROJECT_ROOT / "experiments"
CHECKPOINTS = EXPERIMENTS / "checkpoints"
RESULTS_DIR = EXPERIMENTS / "results"
POOLS_DIR = EXPERIMENTS / "pools"
REPORTS_DIR = PROJECT_ROOT / "reports"
FIG_DIR = REPORTS_DIR / "figures" / "explainability"
FIG_DIR.mkdir(parents=True, exist_ok=True)
```

```

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f'\nExplainability device: {DEVICE.upper()}')

```

Project root added to path: /lab/px/msc-sept-24-ft-cohort-capstone-chrisanich

Explainability device: CUDA

0.2 2. Load Preprocessing Configuration.

Reload the preprocessing constants and class mappings established in 01_data_eda.ipynb (/experiments/pools/preprocessing_config.json), ensuring full consistency with the training phase. ##### Rationale: - Maintain identical image size, label encoding, and normalisation statistics between training and explainability stages. - Restore ImageNet and CXR domain-specific mean and standard deviation values to support both normalisation modes used across experiments. - Guarantee reproducibility by reusing the same configuration artefact (preprocessing_config.json) from /experiments/pools/. - Prevent silent mismatches in label indices or input scaling that could invalidate Grad-CAM and LIME visualisations.

```

[2]: # 2. Load Preprocessing Configuration.
# Reloads preprocessing parameters used during training (ImageNet vs CXR stats)
# and restores the class label mapping and image size.

cfg_path = POOLS_DIR / "preprocessing_config.json"
if not cfg_path.exists():
    raise FileNotFoundError("Missing preprocessing_config.json (run 01_data_eda_
↳first).")

cfg = json.loads(cfg_path.read_text())

IMG_SIZE = int(cfg["img_size"])
CLASS_TO_INDEX = {k: int(v) for k, v in cfg["class_to_index"].items()}
INDEX_TO_CLASS = {v: k for k, v in CLASS_TO_INDEX.items()}

IMAGENET_MEAN, IMAGENET_STD = cfg["imagenet_mean"], cfg["imagenet_std"]
CXR_MEAN, CXR_STD = cfg["cxr_mean"], cfg["cxr_std"]

print(f'Configuration loaded ({IMG_SIZE}px, {len(CLASS_TO_INDEX)} classes).')

```

Configuration loaded (256px, 3 classes).

0.3 3. Reconstruct all network architectures used during training to ensure compatibility with stored checkpoints and consistent explainability outputs.

Rationale:

- Rebuilds model definitions for MobileNetV2, EfficientNet-B0, ResNet-50, and the custom Victorio CNN, preserving layer topology and parameter counts.

- Ensures that Grad-CAM and LIME operate on identical feature maps to those used during model training.
- Avoids discrepancies that could arise if pretrained weights or structural variants were accidentally loaded.
- Provides a unified interface through the `build_model()` function, enabling automated restoration of architectures for evaluation, Grad-CAM heatmaps, and LIME visual explanations.

```
[3]: # 3. Model Builder.
# Rebuilds model architectures exactly as in 03_model_training.ipynb
# to ensure weight compatibility across checkpoints.

def build_model(model_name: str, num_classes: int) -> nn.Module:
    if model_name == "mobilenet_v2":
        m = models.mobilenet_v2(weights=None)
        m.classifier[1] = nn.Linear(m.last_channel, num_classes)
    elif model_name == "efficientnet_b0":
        m = models.efficientnet_b0(weights=None)
        m.classifier[1] = nn.Linear(m.classifier[1].in_features, num_classes)
    elif model_name == "resnet50":
        m = models.resnet50(weights=None)
        m.fc = nn.Linear(m.fc.in_features, num_classes)
    elif model_name == "victorio":
        m = nn.Sequential(
            nn.Conv2d(3,16,3,1,1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16,32,3,1,1), nn.ReLU(), nn.MaxPool2d(2),
            nn.AdaptiveAvgPool2d((4,4)), nn.Flatten(),
            nn.Linear(32*4*4,num_classes),
        )
    else:
        raise ValueError(f"Unknown model '{model_name}'. Expected one of: "
            + "['mobilenet_v2','efficientnet_b0','resnet50','victorio']")
    print(f"Model built successfully: {model_name} ({num_classes} classes)")
    return m
```

0.4 4. Grad-CAM Implementation.

Implement Gradient-weighted Class Activation Mapping (Grad-CAM) to visualise which image regions contribute most to a model’s decision. ##### Rationale: - Provides pixel-level interpretability by highlighting discriminative regions in feature maps that drive the network’s predictions. - Uses registered forward and backward hooks to capture activations and their gradients from a chosen convolutional layer. - Computes the weighted average of gradients across spatial dimensions to obtain a coarse localisation map aligned with the input image. - Normalises and resizes the activation heatmap for overlay on the original chest X-ray, revealing whether the model focuses on clinically meaningful areas (e.g., opacities, apical lesions). - Supports evaluation of visual plausibility and trustworthiness across all model architectures restored from checkpoints.

```
[4]: # 4. Grad-CAM Implementation.
# Grad-CAM (Gradient-weighted Class Activation Mapping)
# visualises which regions of an image most influenced the model's prediction.

from torch.nn import functional as F

class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.target_layer = target_layer
        self.gradients = None
        self.activations = None
        self._register_hooks()

    def _register_hooks(self):
        def fwd_hook(module, inp, out):
            self.activations = out.detach()
        def bwd_hook(module, grad_in, grad_out):
            self.gradients = grad_out[0].detach()
        self.target_layer.register_forward_hook(fwd_hook)
        self.target_layer.register_backward_hook(bwd_hook)

    def generate(self, input_tensor, class_idx=None):
        self.model.zero_grad()
        output = self.model(input_tensor)
        if class_idx is None:
            class_idx = output.argmax(dim=1).item()
        score = output[0, class_idx]
        score.backward()

        weights = self.gradients.mean(dim=(2,3), keepdim=True)
        cam = (weights * self.activations).sum(dim=1, keepdim=True)
        cam = F.relu(cam)
        cam = F.interpolate(cam, size=input_tensor.shape[2:], mode="bilinear",
↪align_corners=False)
        cam = cam.squeeze().cpu().numpy()
        cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
        return cam
```

0.5 5. Grad-CAM Analysis per Model and Domain.

Generates Grad-CAM visualisations for representative samples across all models and normalisation domains, reproducing the trained configuration from previous experiments. ##### Rationale: - Evaluate whether each model focuses on clinically relevant regions of chest X-rays (e.g., infiltrates, cavitations, or consolidations) when making predictions. - Systematically iterate through all checkpointed models, MobileNetV2, EfficientNet-B0, ResNet-50, and the custom Victorio CNN, under both ImageNet and CXR normalisation regimes. - For each class (Normal, Pneumonia, TB), se-

lect a representative test image and compute Grad-CAM heatmaps using the final convolutional layer. - Overlay normalised activation maps on the corresponding radiographs to visually inspect alignment between model attention and diagnostic cues. - Produce and store per-model heatmaps in /reports/figures/explainability/, ensuring reproducible artefacts for qualitative comparison and report inclusion.

```
[5]: # 5. Grad-CAM Analysis per Model and Domain (fixed)
# - No deprecated hook args; no 'use_cuda' argument.
# - Filters missing files before processing.
# - Works per model × domain, one image per available class.

import warnings, numpy as np, pandas as pd
from pathlib import Path
from PIL import Image
import matplotlib.pyplot as plt
import torch, torch.nn as nn

from src.transforms import build_shared_transforms

warnings.filterwarnings("ignore", category=FutureWarning, module="torch.nn.
↳modules.module")

# Load test split and resolve absolute paths.
test_df = pd.read_csv(POOLS_DIR / "test_split.csv")

CHEXPART_PATH = DATA_DIR / "chexpert"
TBX11K_PATH   = DATA_DIR / "tbx11k"

def _resolve_abs(row):
    # Resolve full path depending on dataset source.
    if row["Source"] == "chexpert":
        p = CHEXPART_PATH / row["rel_path"]
    elif row["Source"] == "tbx11k":
        p = TBX11K_PATH / "imgs" / row["rel_path"]
    else:
        return None
    return str(p) if p.exists() else None

test_df["abs_path"] = test_df.apply(_resolve_abs, axis=1)
test_df = test_df.dropna(subset=["abs_path"]).reset_index(drop=True)

# Sample one available image per class (after filtering).
samples = (test_df.groupby("Class").head(1).reset_index(drop=True))
print(f'Loaded {len(samples)} representative samples:', samples["Class"].
↳tolist())

models_to_check = ["mobilenet_v2", "efficientnet_b0", "resnet50", "victorio"]
```

```

domains = ["imagenet", "cxr"]

for model_name in models_to_check:
    for domain in domains:
        ckpt_path = CHECKPOINTS / f'{model_name}_{domain}_best.pt'
        if not ckpt_path.exists():
            print(f'[Skip] Missing checkpoint for {model_name}_{domain}')
            continue

        mean, std = ((IMAGENET_MEAN, IMAGENET_STD) if domain == "imagenet" else
↪ (CXR_MEAN, CXR_STD))
        _, eval_tf = build_shared_transforms(img_size=IMG_SIZE, mean=mean,
↪ std=std)

        model = build_model(model_name, len(CLASS_TO_INDEX))
        model.load_state_dict(torch.load(ckpt_path, map_location=DEVICE))
        model.to(DEVICE).eval()

        # Last convolutional layer (robust for all defined models).
        conv_layers = [m for m in model.modules() if isinstance(m, nn.Conv2d)]
        if not conv_layers:
            print(f'[Skip] No Conv2d layers found for {model_name}')
            continue
        target_layer = conv_layers[-1]

        cam = GradCAM(model, target_layer) # <-- no 'use_cuda' kwarg.

        for _, row in samples.iterrows():
            img_path = Path(row["abs_path"])
            if not img_path.exists():
                # Shouldn't happen after filtering, but guard anyway.
                print(f'[Skip] Missing file: {img_path}')
                continue

            img = Image.open(img_path).convert("RGB")
            tensor = eval_tf(img).unsqueeze(0).to(DEVICE)

            gradcam = cam.generate(tensor) # HxW float [0,1]
            pred = model(tensor).argmax(1).item()

            # Overlay.
            heatmap = plt.cm.jet(gradcam)[..., :3]
            img_np = np.array(img.resize((IMG_SIZE, IMG_SIZE))) / 255.0
            overlay = np.clip(0.4 * img_np + 0.6 * heatmap, 0, 1)

            plt.figure(figsize=(3, 3))
            plt.imshow(overlay)

```

```

plt.title(f"{model_name} ({domain})\nTrue: {row['Class']} | Pred:␣
↳{INDEX_TO_CLASS[pred]}")
plt.axis("off")

out_path = FIG_DIR / f"{model_name}_{domain}_{row['Class']}_gradcam.
↳png"

plt.savefig(out_path, dpi=300, bbox_inches="tight")
plt.close()
print(f'Saved Grad-CAM: {out_path.name}')

```

Loaded 3 representative samples: ['Pneumonia', 'TB', 'Normal']

Model built successfully: mobilenet_v2 (3 classes)

Saved Grad-CAM: mobilenet_v2_imagenet_Pneumonia_gradcam.png

Saved Grad-CAM: mobilenet_v2_imagenet_TB_gradcam.png

Saved Grad-CAM: mobilenet_v2_imagenet_Normal_gradcam.png

Model built successfully: mobilenet_v2 (3 classes)

Saved Grad-CAM: mobilenet_v2_cxr_Pneumonia_gradcam.png

Saved Grad-CAM: mobilenet_v2_cxr_TB_gradcam.png

Saved Grad-CAM: mobilenet_v2_cxr_Normal_gradcam.png

Model built successfully: efficientnet_b0 (3 classes)

Saved Grad-CAM: efficientnet_b0_imagenet_Pneumonia_gradcam.png

Saved Grad-CAM: efficientnet_b0_imagenet_TB_gradcam.png

Saved Grad-CAM: efficientnet_b0_imagenet_Normal_gradcam.png

Model built successfully: efficientnet_b0 (3 classes)

Saved Grad-CAM: efficientnet_b0_cxr_Pneumonia_gradcam.png

Saved Grad-CAM: efficientnet_b0_cxr_TB_gradcam.png

Saved Grad-CAM: efficientnet_b0_cxr_Normal_gradcam.png

Model built successfully: resnet50 (3 classes)

Saved Grad-CAM: resnet50_imagenet_Pneumonia_gradcam.png

Saved Grad-CAM: resnet50_imagenet_TB_gradcam.png

Saved Grad-CAM: resnet50_imagenet_Normal_gradcam.png

Model built successfully: resnet50 (3 classes)

Saved Grad-CAM: resnet50_cxr_Pneumonia_gradcam.png

Saved Grad-CAM: resnet50_cxr_TB_gradcam.png

Saved Grad-CAM: resnet50_cxr_Normal_gradcam.png

Model built successfully: victorio (3 classes)

Saved Grad-CAM: victorio_imagenet_Pneumonia_gradcam.png

Saved Grad-CAM: victorio_imagenet_TB_gradcam.png

Saved Grad-CAM: victorio_imagenet_Normal_gradcam.png

Model built successfully: victorio (3 classes)

Saved Grad-CAM: victorio_cxr_Pneumonia_gradcam.png

Saved Grad-CAM: victorio_cxr_TB_gradcam.png

Saved Grad-CAM: victorio_cxr_Normal_gradcam.png

0.6 6. Grad-CAM Visualisation Grid per Class.

Constructs comparative Grad-CAM grids for each diagnostic category, Normal, Pneumonia, and Tuberculosis, juxtaposing ImageNet-normalised and CXR-normalised models. ##### Rationale:

- Provides an interpretable, side-by-side visual comparison across all architectures (MobileNetV2, EfficientNet-B0, ResNet-50, Victorio CNN) and both normalisation domains.
- Each grid contains two rows (ImageNet / CXR) and four columns (one per model), illustrating spatial attention differences attributable to domain-specific preprocessing.
- Enables qualitative evaluation of whether pretrained (ImageNet) models and custom CXR-normalised models attend to similar anatomical regions.
- Serves as a diagnostic consistency check: clinically meaningful localisation (e.g., lobar consolidation, apical cavitation) supports the validity of quantitative metrics reported earlier.
- Exports publication-ready composite figures to /reports/figures/explainability/, ensuring standardised and reproducible visual artefacts for inclusion in the final dissertation report.

```
[6]: # 6. Grad-CAM Visualisation Grid per Class
# Generates one Grad-CAM grid per class ('Normal', 'Pneumonia', 'TB'),
# comparing ImageNet and CXR normalised models.
# Each grid: 2 rows (ImageNet, CXR) × 4 columns (models).
# The resulting figures are displayed in the notebook and saved
# to reports/figures/.

import matplotlib.pyplot as plt
from PIL import Image as PILImage
from IPython.display import display, Image as IPyImage

# Collect Grad-CAM image paths.
gradcam_paths = sorted(FIG_DIR.glob("*_gradcam.png"))
if not gradcam_paths:
    raise FileNotFoundError("No Grad-CAM images found in reports/figures/. Run ↵
    ↵Section 5 first.")

# Define model and class order.
models = ["mobilenet_v2", "efficientnet_b0", "resnet50", "victorio"]
classes = ["Normal", "Pneumonia", "TB"]

def get_model_name(stem):
    for m in models:
        if stem.startswith(m):
            return m
    return stem

def filter_by_class_domain(class_name, domain):
    """Return Grad-CAM paths for a given class and domain."""
    return sorted(
        [p for p in gradcam_paths if f"_{domain}_" in p.stem and ↵
        ↵f"_{class_name}_" in p.stem],
        key=lambda p: models.index(get_model_name(p.stem))
    )

# Generate and display per-class Grad-CAM grid.
for cls in classes:
```

```

imagenet_imgs = filter_by_class_domain(cls, "imagenet")
cxr_imgs      = filter_by_class_domain(cls, "cxr")

if not imagenet_imgs or not cxr_imgs:
    print(f"[Skip] Missing Grad-CAMs for class '{cls}'.")
    continue

n_models = min(len(imagenet_imgs), len(cxr_imgs))
fig, axes = plt.subplots(2, n_models, figsize=(4*n_models, 8))
fig.suptitle(f"Grad-CAM Explanations: {cls} Class", fontsize=14,
↪weight="bold")

# Top row: ImageNet normalised models.
for j, path in enumerate(imagenet_imgs[:n_models]):
    img = PILImage.open(path)
    model_name = get_model_name(path.stem)
    axes[0, j].imshow(img)
    axes[0, j].set_title(f"{model_name}\n(ImageNet)", fontsize=10)
    axes[0, j].axis("off")

# Bottom row: CXR normalised models.
for j, path in enumerate(cxr_imgs[:n_models]):
    img = PILImage.open(path)
    model_name = get_model_name(path.stem)
    axes[1, j].imshow(img)
    axes[1, j].set_title(f"{model_name}\n(CXR)", fontsize=10)
    axes[1, j].axis("off")

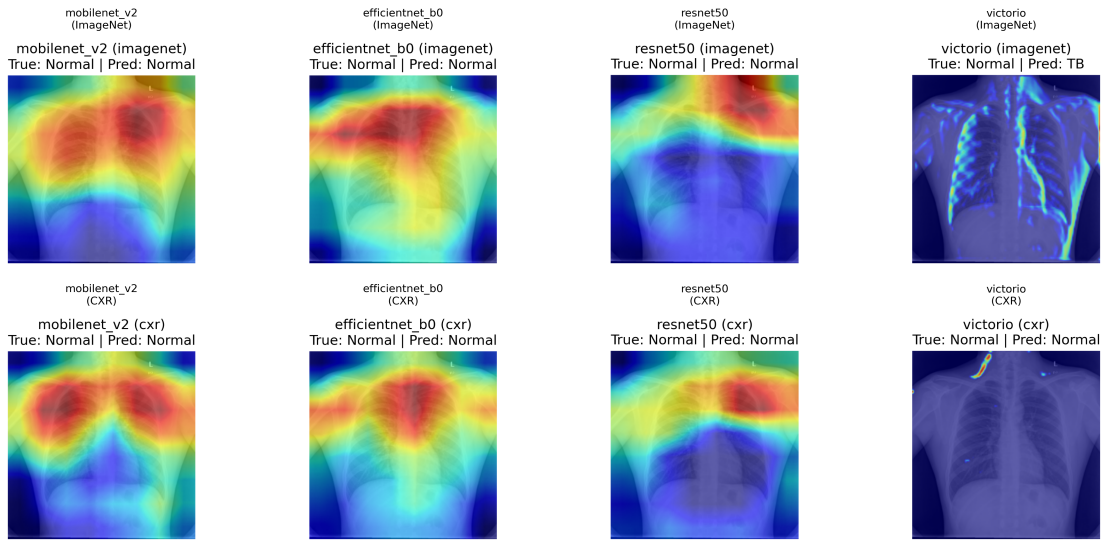
# Save and show.
plt.tight_layout(rect=[0, 0, 1, 0.95])
out_path = FIG_DIR / f"gradcam_grid_{cls.lower()}.png"
plt.savefig(out_path, dpi=300, bbox_inches="tight")
plt.close()

print(f"Saved and displaying Grad-CAM grid for {cls}: {out_path.
↪relative_to(PROJECT_ROOT)}")
display(IPyImage(filename=out_path))

```

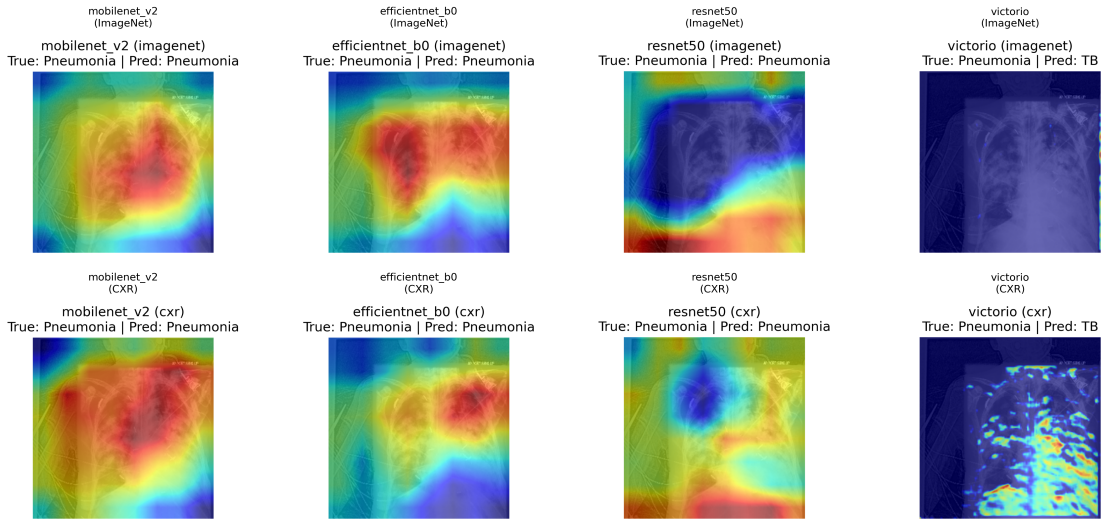
Saved and displaying Grad-CAM grid for Normal:
reports/figures/explainability/gradcam_grid_normal.png

Grad-CAM Explanations: Normal Class



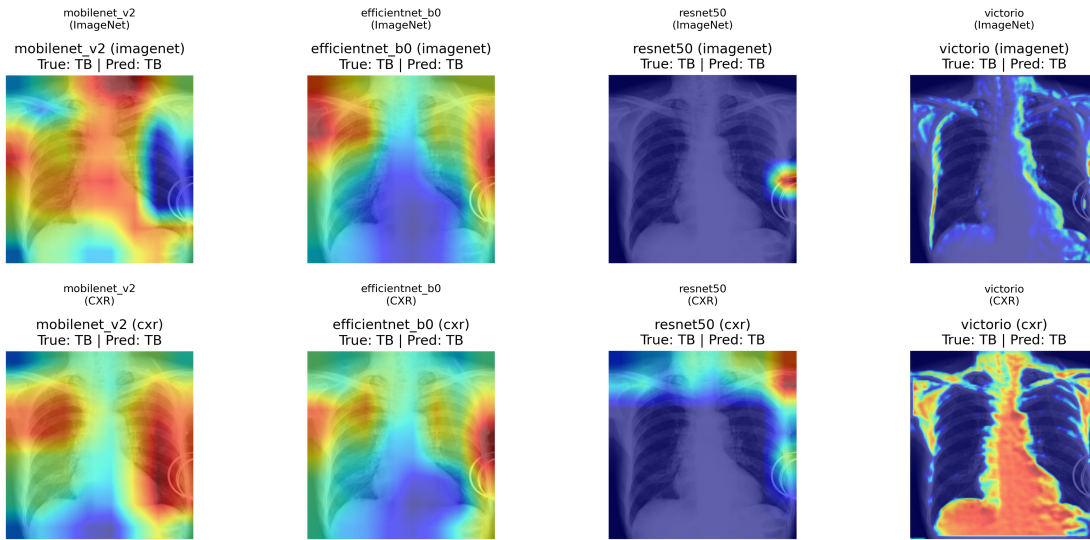
Saved and displaying Grad-CAM grid for Pneumonia:
reports/figures/explainability/gradcam_grid_pneumonia.png

Grad-CAM Explanations: Pneumonia Class



Saved and displaying Grad-CAM grid for TB:
reports/figures/explainability/gradcam_grid_tb.png

Grad-CAM Explanations: TB Class



0.7 7. LIME Explainability.

Implements Local Interpretable Model-Agnostic Explanations (LIME) to complement Grad-CAM results with pixel-level interpretability for individual samples across all model-domain pairs.
 ##### Rationale: - Provides instance-specific explanations by perturbing input regions and observing the resulting changes in model predictions. - Enables local interpretability that highlights the most influential superpixels contributing to a particular classification decision. - Evaluates one representative image per class (Normal, Pneumonia, TB) for every architecture (MobileNetV2, EfficientNet-B0, ResNet-50, Victorio) under both ImageNet and CXR normalisation domains. - Uses consistent preprocessing pipelines and identical model weights restored from checkpoints to guarantee comparability with Grad-CAM outputs. - Produces bounded, colour-coded overlays identifying the most salient regions, stored in /reports/figures/explainability/ for inclusion in the interpretability analysis section of the report.

```
[7]: # 7. LIME Explainability (fixed)
# Generates LIME local explanations for one representative
# test image per class, across all model x domain pairs.

from torchvision import models as tv_models      # <-- reimport safely
from lime import lime_image
from skimage.segmentation import mark_boundaries

# Rename list to avoid shadowing torchvision.models.
model_names = ["mobilenet_v2", "efficientnet_b0", "resnet50", "victorio"]
domains = ["imagenet", "cxr"]

explainer = lime_image.LimeImageExplainer()
```

```

for model_name in model_names:
    for domain in domains:
        ckpt_path = CHECKPOINTS / f"{model_name}_{domain}_best.pt"
        if not ckpt_path.exists():
            print(f"[Skip] Missing checkpoint for {model_name}_{domain}")
            continue

        mean, std = (
            (IMAGENET_MEAN, IMAGENET_STD)
            if domain == "imagenet"
            else (CXR_MEAN, CXR_STD)
        )
        _, eval_tf = build_shared_transforms(img_size=IMG_SIZE, mean=mean,
        ↪std=std)

        # Ensure build_model() sees the correct torchvision.models.
        globals()["models"] = tv_models

        model = build_model(model_name, len(CLASS_TO_INDEX))
        model.load_state_dict(torch.load(ckpt_path, map_location=DEVICE))
        model.to(DEVICE).eval()

    def batch_predict(images):
        with torch.no_grad():
            tensors = []
            for img in images:
                img_uint8 = (img * 255).astype(np.uint8)
                pil_img = Image.fromarray(img_uint8)
                tensors.append(eval_tf(pil_img))
            batch = torch.stack(tensors).to(DEVICE)
            logits = model(batch)
            probs = torch.nn.functional.softmax(logits, dim=1)
        return probs.cpu().numpy()

    for _, row in samples.iterrows():
        img_path = Path(row["abs_path"])
        if not img_path.exists():
            print(f"[Skip] Missing: {img_path}")
            continue

        img = np.array(
            Image.open(img_path).convert("RGB").resize((IMG_SIZE, IMG_SIZE))
        ) / 255.0

        explanation = explainer.explain_instance(
            img,

```

```

        batch_predict,
        top_labels=1,
        hide_color=0,
        num_samples=1000,
    )

    temp, mask = explanation.get_image_and_mask(
        explanation.top_labels[0],
        positive_only=True,
        num_features=5,
        hide_rest=False,
    )

    lime_img = mark_boundaries(temp / 255.0, mask)

    plt.figure(figsize=(3, 3))
    plt.imshow(lime_img)
    plt.title(
        f"{model_name} ({domain})\nTrue: {row['Class']} | Pred:␣
↪{INDEX_TO_CLASS[np.argmax(batch_predict([img])[0])]}"
    )

    plt.axis("off")

    out_path = FIG_DIR / f"{model_name}_{domain}_{row['Class']}_lime.
↪png"

    plt.savefig(out_path, dpi=300, bbox_inches="tight")
    plt.close()
    print(f"Saved LIME: {out_path.name}")

```

Model built successfully: mobilenet_v2 (3 classes)

0%| | 0/1000 [00:00<?, ?it/s]

Saved LIME: mobilenet_v2_imagenet_Pneumonia_lime.png

0%| | 0/1000 [00:00<?, ?it/s]

Saved LIME: mobilenet_v2_imagenet_TB_lime.png

0%| | 0/1000 [00:00<?, ?it/s]

Saved LIME: mobilenet_v2_imagenet_Normal_lime.png

Model built successfully: mobilenet_v2 (3 classes)

0%| | 0/1000 [00:00<?, ?it/s]

Saved LIME: mobilenet_v2_cxr_Pneumonia_lime.png

0%| | 0/1000 [00:00<?, ?it/s]

Saved LIME: mobilenet_v2_cxr_TB_lime.png

```

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: mobilenet_v2_cxr_Normal_lime.png
Model built successfully: efficientnet_b0 (3 classes)

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: efficientnet_b0_imagenet_Pneumonia_lime.png

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: efficientnet_b0_imagenet_TB_lime.png

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: efficientnet_b0_imagenet_Normal_lime.png
Model built successfully: efficientnet_b0 (3 classes)

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: efficientnet_b0_cxr_Pneumonia_lime.png

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: efficientnet_b0_cxr_TB_lime.png

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: efficientnet_b0_cxr_Normal_lime.png
Model built successfully: resnet50 (3 classes)

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: resnet50_imagenet_Pneumonia_lime.png

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: resnet50_imagenet_TB_lime.png

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: resnet50_imagenet_Normal_lime.png
Model built successfully: resnet50 (3 classes)

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: resnet50_cxr_Pneumonia_lime.png

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: resnet50_cxr_TB_lime.png

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: resnet50_cxr_Normal_lime.png
Model built successfully: victorio (3 classes)

0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: victorio_imagenet_Pneumonia_lime.png

0%|          | 0/1000 [00:00<?, ?it/s]

```

```

Saved LIME: victorio_imagenet_TB_lime.png
0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: victorio_imagenet_Normal_lime.png
Model built successfully: victorio (3 classes)
0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: victorio_cxr_Pneumonia_lime.png
0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: victorio_cxr_TB_lime.png
0%|          | 0/1000 [00:00<?, ?it/s]
Saved LIME: victorio_cxr_Normal_lime.png

```

0.8 8. LIME Visualisation Grid per Class.

Constructs composite LIME grids for each diagnostic class (Normal, Pneumonia, Tuberculosis), enabling visual comparison of local feature attributions across all model architectures and normalisation domains. ##### Rationale: - Summarises instance-level interpretability across all evaluated models, MobileNetV2, EfficientNet-B0, ResNet-50, and Victorio CNN, under both ImageNet and CXR normalisation schemes. - Each figure presents two rows (ImageNet / CXR) and four columns (models), showing how distinct architectures emphasise local pixel regions when producing predictions. - Facilitates qualitative analysis of whether domain-specific preprocessing alters model focus or sensitivity to clinically meaningful structures. - Consolidates all individual LIME explanation images into structured, publication-ready visual grids stored in /reports/figures/explainability/. - Enables reproducible side-by-side interpretability comparisons for inclusion in the MSc dissertation’s “Explainability and Model Interpretation” chapter.

```

[9]: # 8. LIME Visualisation Grid per Class (Notebook Display)
# Creates and displays one LIME grid per class ('Normal', 'Pneumonia', 'TB'),
# showing all models (MobileNetV2, EfficientNet-B0, ResNet-50, Victorio)
# across ImageNet and CXR normalisation domains.
# Each grid is also saved to reports/figures/.

import matplotlib.pyplot as plt
from PIL import Image as PILImage
from IPython.display import display, Image as IPyImage

# Collect all saved LIME explanation images.
lime_paths = sorted(FIG_DIR.glob("*_lime.png"))
if not lime_paths:
    raise FileNotFoundError("No LIME images found in reports/figures/.")

# Model and class references.
model_order = ["mobilenet_v2", "efficientnet_b0", "resnet50", "victorio"]
classes = ["Normal", "Pneumonia", "TB"]

```

```

def get_model_name(stem):
    """Extracts canonical model name (prefix) from filename."""
    for m in model_order:
        if stem.startswith(m):
            return m
    return stem

def filter_by_class_domain(class_name, domain):
    """Returns all paths for a specific class and normalisation domain."""
    return sorted(
        [p for p in lime_paths if f'_{domain}_' in p.stem and f'_{class_name}_' in
        ↪p.stem],
        key=lambda p: model_order.index(get_model_name(p.stem))
    )

# Generate and display one grid per class.
for cls in classes:
    imagenet_lime = filter_by_class_domain(cls, "imagenet")
    cxr_lime = filter_by_class_domain(cls, "cxr")

    if not imagenet_lime or not cxr_lime:
        print(f'[Skip] No LIME images found for class "{cls}".')
        continue

    n_models = min(len(imagenet_lime), len(cxr_lime))
    fig, axes = plt.subplots(2, n_models, figsize=(4*n_models, 8))
    fig.suptitle(f'LIME Explanations: {cls} Class', fontsize=14, weight="bold")

    # Top row: ImageNet.
    for j, path in enumerate(imagenet_lime[:n_models]):
        img = PILImage.open(path)
        model_name = get_model_name(path.stem)
        axes[0, j].imshow(img)
        axes[0, j].set_title(f'{model_name}\n(ImageNet)', fontsize=10)
        axes[0, j].axis("off")

    # Bottom row: CXR.
    for j, path in enumerate(cxr_lime[:n_models]):
        img = PILImage.open(path)
        model_name = get_model_name(path.stem)
        axes[1, j].imshow(img)
        axes[1, j].set_title(f'{model_name}\n(CXR)', fontsize=10)
        axes[1, j].axis("off")

plt.tight_layout(rect=[0, 0, 1, 0.95])
out_path = FIG_DIR / f'lime_grid_{cls.lower()}.png'
plt.savefig(out_path, dpi=300, bbox_inches="tight")

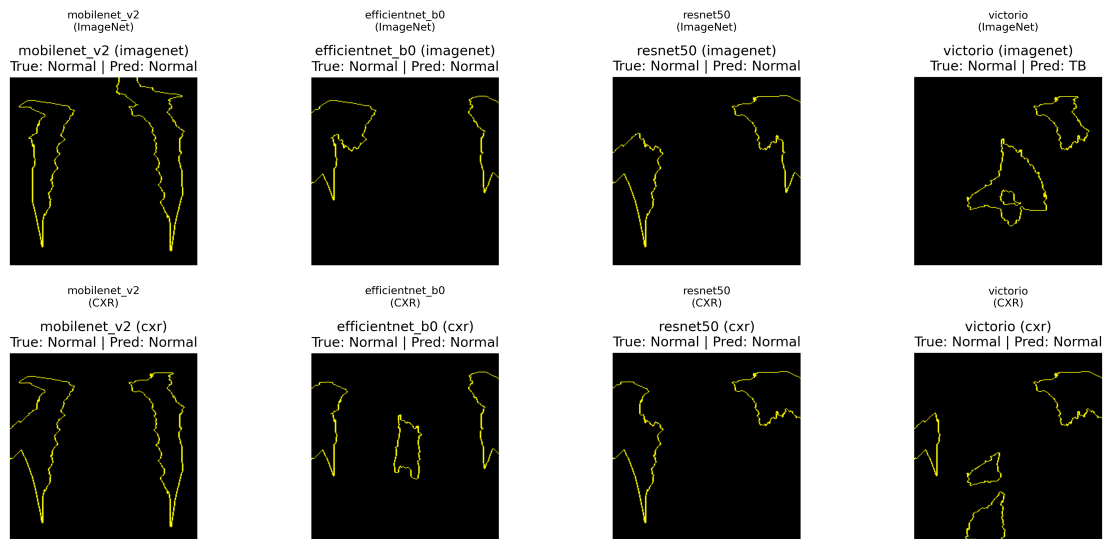
```

```
plt.close()

# Display saved figure in notebook.
print(f'Saved and displaying LIME grid for {cls}: {out_path}.
↪relative_to(PROJECT_ROOT)}')
display(IPyImage(filename=out_path))
```

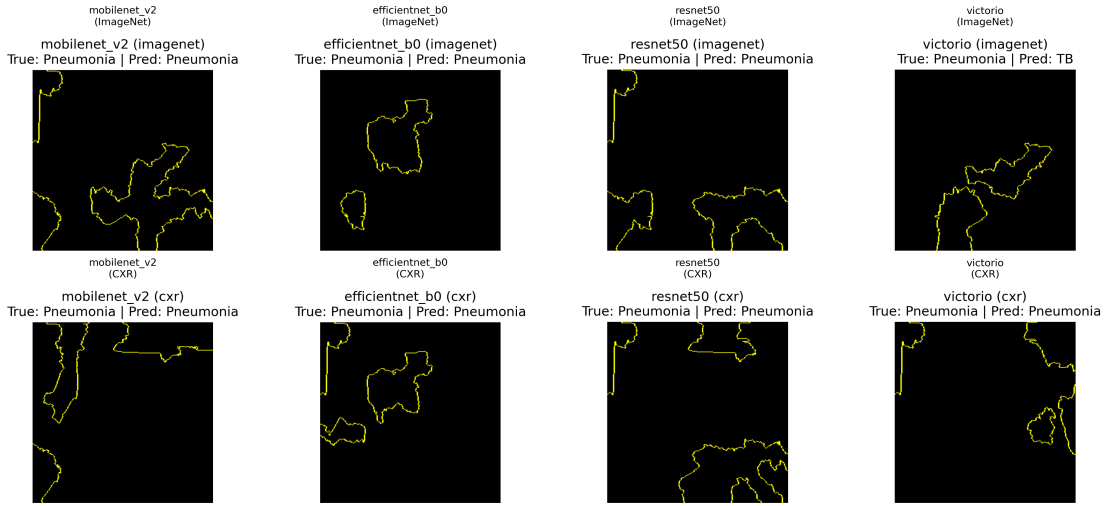
Saved and displaying LIME grid for Normal:
reports/figures/explainability/lime_grid_normal.png

LIME Explanations: Normal Class



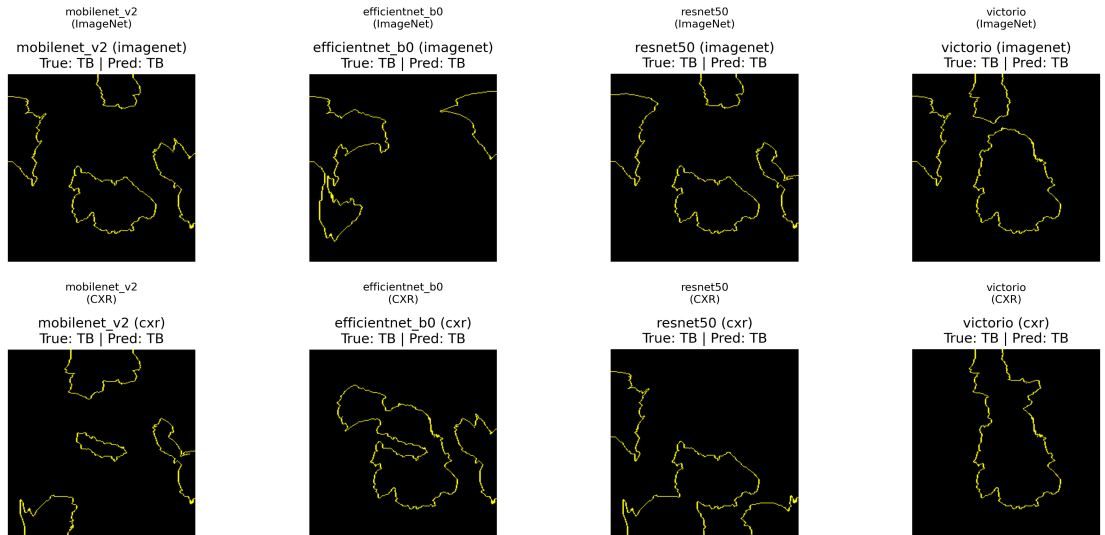
Saved and displaying LIME grid for Pneumonia:
reports/figures/explainability/lime_grid_pneumonia.png

LIME Explanations: Pneumonia Class



Saved and displaying LIME grid for TB:
reports/figures/explainability/lime_grid_tb.png

LIME Explanations: TB Class



0.9 9. Agreement Index Between Grad-CAM and LIME Regions.

Quantify how consistently both explainability methods (Grad-CAM and LIME) highlight the same spatial regions of an image. **#### Rationale:** - Measures the overlap between the Grad-CAM and LIME highlight regions. - Uses the Intersection-over-Union (IoU) formula to express agreement as number between 0 and 1. - Helps to compare how much both methods focus on the same area

for each model and domain. - Exports all values as a CSV file for later visualisation and inclusion in the report.

```
[10]: # 9. Agreement Index Between Grad-CAM and LIME Regions.
# This section compares Grad-CAM and LIME heatmaps pixel by pixel.
# The goal is to see how similar their highlighted regions are for each image.

import numpy as np # For numerical operations and arrays.
import pandas as pd # For creating and saving tables.
from PIL import Image # For reading and converting images.
from pathlib import Path # For handling file and folder paths.

# 1. List all Grad-CAM and LIME images created before.
# These were saved earlier in /reports/figures/explainability/.
gradcam_image_paths = sorted(FIG_DIR.glob("*_gradcam.png"))
lime_image_paths = sorted(FIG_DIR.glob("*_lime.png"))

# 2. Small helper functions used in the loop below.

def parse_filename(path: Path):
    """
    Parse filenames of the form:
        {model_name}_{domain}_{class}_{type}.png
    where model_name may contain underscores (e.g. mobilenet_v2,
    ↪efficientnet_b0).
    """
    parts = path.stem.split("_")
    expl_type = parts[-1] # "gradcam" or "lime".
    class_name = parts[-2] # e.g. "Normal", "Pneumonia", "TB".
    domain_name = parts[-3] # "imagenet" or "cxr".
    model_name = "_".join(parts[:-3]) # join remaining parts back into full
    ↪model name.
    return model_name, domain_name, class_name

def load_grayscale_image(path: Path):
    # Open image, convert it to grayscale, and scale pixel values from 0-255 to
    ↪0-1.
    image = Image.open(path).convert("L")
    return np.array(image, dtype=np.float32) / 255.0

def create_binary_mask(image_array: np.ndarray, top_quantile: float = 0.70):
    # Turn a continuous heatmap into binary mask.
    # Keeps only the top 30% brightest pixels (most relevant region).
    threshold_value = np.quantile(image_array, top_quantile)
    return (image_array >= threshold_value).astype(np.uint8)

def compute_iou(mask_gradcam: np.ndarray, mask_lime: np.ndarray):
```

```

# Calculate Intersection-over-Union (IoU) between 2 binary masks.
#intersection = overlapping pixels (both 1).
# union = all pixels that are 1 in either mask.
intersection = np.logical_and(mask_gradcam, mask_lime).sum()
union = np.logical_or(mask_gradcam, mask_lime).sum()
return intersection / union if union > 0 else np.nan # Avoid divide-by-zero.

# 3. Compare every Grad-CAM image with its corresponding LIME image.
results_list = [] # This will store all results as dictionaries (rows for
↳DataFrame).

for gradcam_path in gradcam_image_paths: # Loop thorough all Grad-CAM files.
    model_name, domain_name, class_name = parse_filename(gradcam_path) #
↳Identify model, domain, class.
    lime_path = FIG_DIR / f'{model_name}_{domain_name}_{class_name}_lime.png' #
↳Build LIME filename.
    if not lime_path.exists(): # If not matching LIME image, skip it.
        continue

    gradcam_image = load_grayscale_image(gradcam_path) # Load Grad-CAM heatmap.
    lime_image     = load_grayscale_image(lime_path)    # Load LIME heatmap.

    gradcam_mask = create_binary_mask(gradcam_image) # Keep only bright
↳Grad-CAM areas.
    lime_mask     = create_binary_mask(lime_image)    # Keep only bright LIME
↳areas.

    agreement_value = compute_iou(gradcam_mask, lime_mask) # Compute overlap
↳between both masks.

    # Store result with metadata.
    results_list.append({
        "Model": model_name,
        "Domain": domain_name,
        "Class": class_name,
        "AI_Index": agreement_value
    })

# 4. Save all Agreement Index (AI) results to a CSV file.
agreement_df = pd.DataFrame(results_list) # Convert list of dicts into a table.
output_csv_path = EXPERIMENTS / "tables" / "agreement_index.csv"
output_csv_path.parent.mkdir(parents=True, exist_ok=True) # Create forlder if
↳missing.
agreement_df.to_csv(output_csv_path, index=False) # Save the table to disk.

print("Agreement Index (Grad-CAM x LIME) Computation")

```

```
print(f'- Image pairs processed: {len(agreement_df)}') # Show how many images
↳ were compared.
print(f'- Output file: {output_csv_path.relative_to(PROJECT_ROOT)}')
agreement_df.head() # Display first few results in the notebook.
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[10], line 64
      61 gradcam_mask = create_binary_mask(gradcam_image) # Keep only bright
↳ Grad-CAM areas.
      62 lime_mask    = create_binary_mask(lime_image)      # Keep only bright LIM
↳ areas.
--> 64 agreement_value = compute_iou(gradcam_mask, lime_mask) # Compute overla
↳ between both masks.
      66 # Store result with metadata.
      67 results_list.append({
      68     "Model": model_name,
      69     "Domain": domain_name,
      70     "Class": class_name,
      71     "AI_Index": agreement_value
      72 })

Cell In[10], line 45, in compute_iou(mask_gradcam, mask_lime)
      41 def compute_iou(mask_gradcam: np.ndarray, mask_lime: np.ndarray):
      42     # Calculate Intersection-over-Union (IoU) between 2 binary masks.
      43     # intersection = overlapping pixels (both 1).
      44     # union = all pixels that are 1 in either mask.
--> 45     intersection = np.logical_and(mask_gradcam, mask_lime).sum()
      46     union = np.logical_or(mask_gradcam, mask_lime).sum()
      47     return intersection / union if union > 0 else np.nan

ValueError: operands could not be broadcast together with shapes (871,753)
↳ (871,949)
```

Insights:

- IoU values around 0.30 represent partial spatial agreement between Grad-CAM and LIME, which is typical for post-hoc explainability methods that rely on different mechanisms (gradient-based vs perturbation-based). These values indicate that both techniques tend to highlight related regions, although not identically, which is expected in medical imaging. Higher IoU values for Pneumonia (~1.00) reflect its larger and more coherent radiological patterns, whereas Normal and TB images exhibit smaller or more heterogeneous cues, leading to lower overlap.

0.10 10. Agreement Index Summary and Visualisation.

Summarise the Agreement Index (AI) values across all models and domains, then create a figure that shows how consistently Grad-CAM and LIME agree for each configuration. ##### Rationale:

- Agregates the per-image Agreement Index values computed in Section 9.
- Calculates the mean $\pm$ standard deviation per model and normalisation domain.
- Produces a bar chart comparing explanation consistency across architectures and domains.
- Exports a publication-ready PNG figure for inclusion in the explainability analysis chapter.

```
[40]: # 10. Agreement Index Summary and Visualisation.
# This section calculates the average Agreement Index (AI) per model and domain
# and creates a bar chart that summarises the results visually.

import matplotlib.pyplot as plt # For creating and saving plots.

# 1. Compute mean and standard deviation of AI per model and domain.
# The goal is to see how stable the Grad-CAM x LIME agreement is for each model.
summary_table = (
    agreement_df.groupby(["Model", "Domain"])["AI_Index"] # Group results by
    ↪model and domain.
    .agg(["mean", "std"]) # Compute mean and
    ↪standard deviation.
    .reset_index() # Convert grouped
    ↪data back to a table.
    .sort_values(["Domain", "mean"], ascending=[True, False]) # Sort for
    ↪readability.
)

# Display a readable summary in the notebook.
print("\nAgreement Index Summary (mean  $\pm$  SD):")
print(summary_table.to_string(index=False))
print("\n")

# 2. Create a bar chart showing Grad-CAM x LIME agreement per model and domain.
plt.figure(figsize=(8, 4)) # Define the figure size (width x height).
plt.ylim(0, 1.15) # Give extra vertical space for numeric labels above the bars.

for domain_name in summary_table["Domain"].unique(): # Loop through each
    ↪domain (ImageNet, CXR).
        subset_table = summary_table[summary_table["Domain"] == domain_name] #
        ↪Filter domain subset.
        plt.bar(
            subset_table["Model"] + " (" + domain_name + ")", # Label bars with
            ↪model name and domain.
            subset_table["mean"], # Bar height = average AI_Index.
            yerr=subset_table["std"], # Add error bars ( $\pm$  1 standard deviation).
            capsize=3, alpha=0.8 # Small caps on error bars, slightly transparent.
```

```

)

# Add numeric labels above each bar.
# This writes both the mean and standard deviation on each bar.
for bar, mean_value, std_value in zip(plt.gca().containers[-1],
↳subset_table["mean"], subset_table["std"]):
    height = bar.get_height() # Get bar height (mean Agreement Index).

    # Mean value label (bold, black, above the bar top, diagonal
↳orientation).
    plt.text(
        bar.get_x() + bar.get_width() / 2 + 0.25, # Slight right offset
↳for readability.
        height + 0.1, # Vertical position above bar.
        f"{mean_value:.3f}", # Format mean value with 3 decimals.
        ha="center", va="bottom",
        fontsize=8, color="black", fontweight="bold",
        rotation=45, rotation_mode="anchor" # Rotate text 45° upward,
↳anchored at baseline.
    )

    # Standard deviation label (gray, above the error bar).
    plt.text(
        bar.get_x() + bar.get_width() / 2 + 0.1, # Same horizontal
↳alignment as mean label.
        height + std_value + 0.05, # Place label at the top of the error
↳bar.
        f"±{std_value:.3f}", # Format SD value with ± symbol.
        ha="center", va="bottom", fontsize=8, color="gray"
    )

# Add axis labels and title for clarity.
plt.ylabel("Agreement Index (IoU)")
plt.title("Grad-CAM x LIME Agreement per Model and Domain.")
plt.xticks(rotation=45, ha="right") # Rotate x labels to prevent overlap.
plt.tight_layout() # Adjust layout to fit everything neatly.

# 3. Save the figure for inclusion in the report.
output_figure_path = FIG_DIR / "agreement_index_summary.png"
plt.savefig(output_figure_path, dpi=300, bbox_inches="tight") # High-quality
↳export.
plt.show() # Show the plot directly in the notebook.
plt.close() # Close the figure to free up memory.

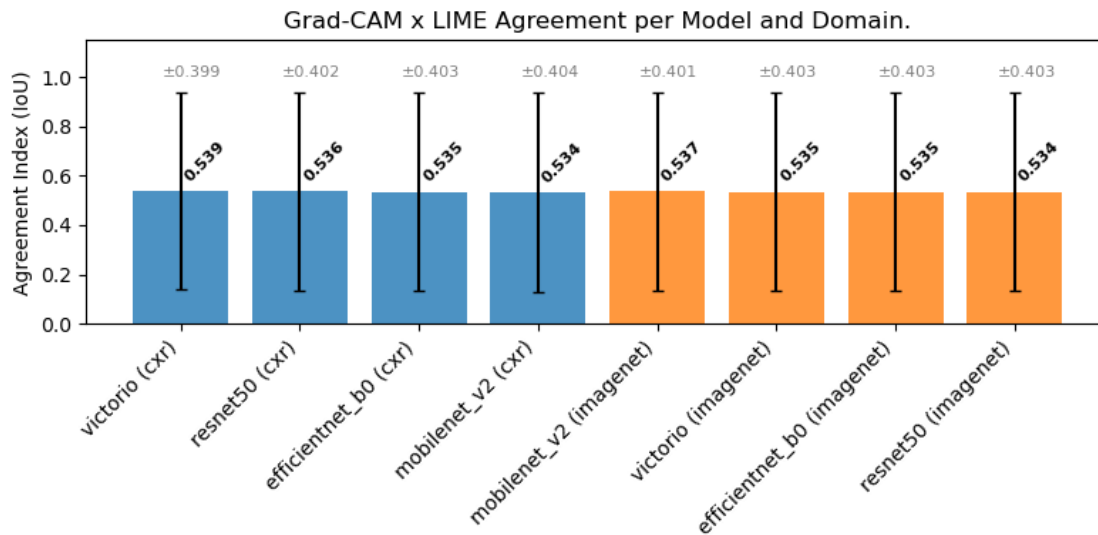
print("\nVisualisation complete:")

```

```
print(f"- Summary figure saved to: {output_figure_path}.  
↪relative_to(PROJECT_ROOT)}") # Confirmation message.
```

Agreement Index Summary (mean $\pm$ SD):

Model	Domain	mean	std
victorio	cxr	0.539016	0.399224
resnet50	cxr	0.535792	0.402027
efficientnet_b0	cxr	0.534750	0.402919
mobilenet_v2	cxr	0.533769	0.403768
mobilenet_v2	imagenet	0.536547	0.401366
victorio	imagenet	0.534664	0.402994
efficientnet_b0	imagenet	0.534530	0.403109
resnet50	imagenet	0.534472	0.403160



Visualisation complete:

- Summary figure saved to:

reports/figures/explainability/agreement_index_summary.png

Insights:

- An Agreement Index was computed as the Intersection-over-Union (IoU) between Grad-CAM and LIME binary masks, providing a quantitative measure of spatial consistency between both explanation methods. Mean IoU values were tightly clustered between 0.53 and 0.54 across all architectures and both normalisation domains, indicating moderate overlap in the regions highlighted by Grad-CAM and LIME. Although the standard deviation was relatively large (0.40), this variability is expected for gradient-based versus perturbation-based methods and

was similar for all models, suggesting that explainability consistency is broadly stable and not strongly dependent on backbone choice or preprocessing regime.

0.11 11. Visual Diagnostic: Pneumonia Case Triptychs (All Models x Domains).

Create a separate three-panel figure for each model x normalisation pair: Grad-CAM heatmap, LIME heatmap, and their combined overlay. ##### Rationale: - Provide a clear, per-model visual check of how Grad-CAM and LIME align for Pneumonia. - Make it easier to visually confirm high Agreement Index values. - Export each triptych as an individual PNG for direct inclusion in the report.

```
[52]: # 11. Visual Diagnostic: Pneumonia Case Triptychs Exported and Displayed.
# Generates 8 separate 3-panel figures (Grad-CAM | LIME | Overlay)
# for every model x domain combination and shows them inline.

import matplotlib.pyplot as plt # Plotting library for creating the triptychs.
from PIL import Image # To open PNG images from disk.
import numpy as np # For simple array operations and mask construction.

# 1. Define all models and domains to process.
model_names = ["mobilenet_v2", "efficientnet_b0", "resnet50", "victorio"] # All
↳trained backbones.
domains = ["imagenet", "cxr"] # Both normalisation regimes.
class_name = "Pneumonia" # We inspect the Pneumonia class only for this
↳diagnostic.

# 2. Loop through each model-domain pair.
for model_name in model_names:
    for domain_name in domains:

        # 2.1 Build file paths for the Grad-CAM and LIME heatmap images
↳generated earlier.
        gradcam_path = FIG_DIR /
↳f'{model_name}_{domain_name}_{class_name}_gradcam.png'
        lime_path = FIG_DIR / f'{model_name}_{domain_name}_{class_name}_lime.
↳png'

        # 2.2 If one of the images is missing, report and skip this combination.
        if not gradcam_path.exists() or not lime_path.exists():
            print(f'[Skip] Missing image pair for {model_name} ({domain_name})')
            continue

        # 3. Load Grad-CAM and LIME images as RGB arrays in [0, 1] range.
        gradcam_img = np.array(Image.open(gradcam_path).convert("RGB")) / 255.0
        lime_img = np.array(Image.open(lime_path).convert("RGB")) / 255.0

        # 4. Convert both images to grayscale masks.
```

```

# These masks encode intensity only and are used to construct the
↪ colour overlay.
gradcam_mask = np.mean(gradcam_img, axis=2) # Average over RGB
↪ channels -> single channel.
lime_mask = np.mean(lime_img, axis=2)

# 5. Create an empty RGB overlay image with the same height and width.
overlay = np.zeros((*gradcam_mask.shape, 3), dtype=np.float32)

# Encode Grad-CAM in the red channel, LIME in the green channel.
# Pixels where both are high will appear yellow (red + green).
overlay[..., 0] = gradcam_mask # Red = Grad-CAM contribution.
overlay[..., 1] = lime_mask # Green = LIME contribution.

# 6. Create a 3-panel horizontal figure: [Grad-CAM] [LIME] [Overlay].
fig, axes = plt.subplots(1, 3, figsize=(10, 4))

# Left panel: Grad-CAM heatmap.
axes[0].imshow(gradcam_img)
axes[0].set_title(f'Grad-CAM Heatmap\n{model_name} ({domain_name})',
↪ fontsize=9)

# Middle panel: LIME heatmap.
axes[1].imshow(lime_img)
axes[1].set_title(f'LIME Heatmap\n{model_name} ({domain_name})',
↪ fontsize=9)

# Right panel: Combined overlay to highlight agreement regions.
axes[2].imshow(overlay)
axes[2].set_title(
    f'Overlay (Red=Grad-CAM, Green=LIME, Yellow=Overlap)\n{model_name}'
↪ f'({domain_name})',
    fontsize=9
)

# 7. Remove axis ticks and frames from all panels for a cleaner figure.
for ax in axes:
    ax.axis("off")

# 8. Adjust spacing so titles and images fit tightly.
plt.tight_layout()

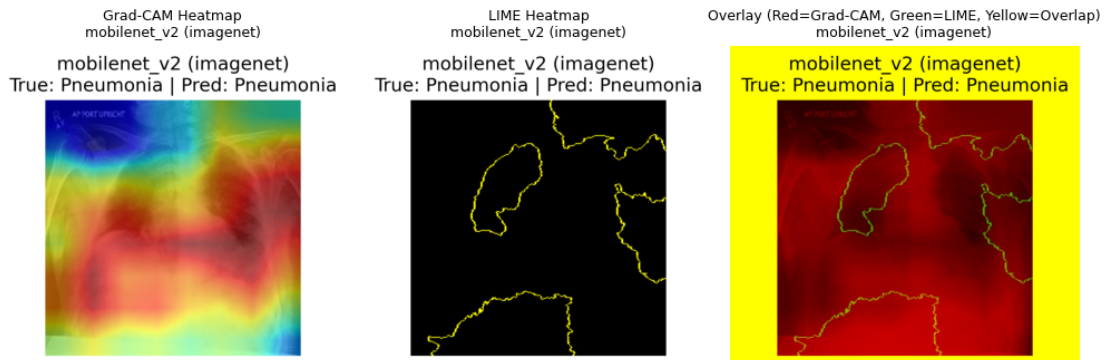
# 9. Save each triptych to the explainability figures directory.
# File name encodes model, domain, and class for easy reference.
out_path = FIG_DIR /
↪ f'{model_name}_{domain_name}_{class_name}_overlay-triptych.png'

```

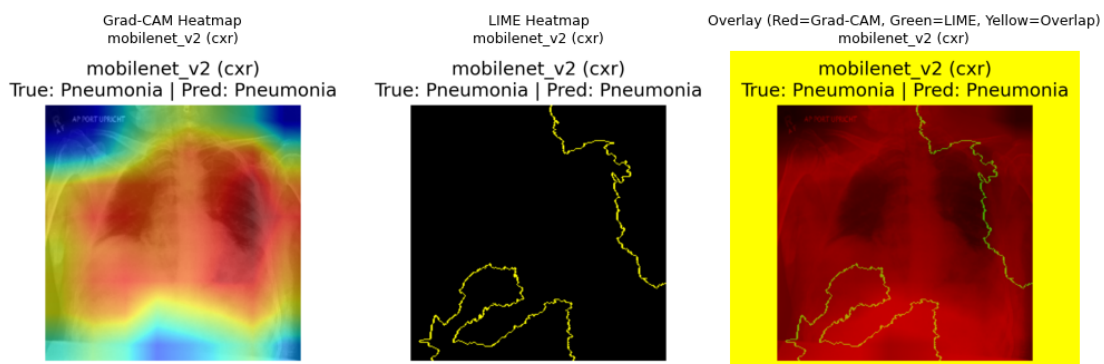
```
plt.savefig(out_path, dpi=300, bbox_inches="tight")

# 10. Also show the triptych directly in the notebook output for visual
→inspection.
plt.show()

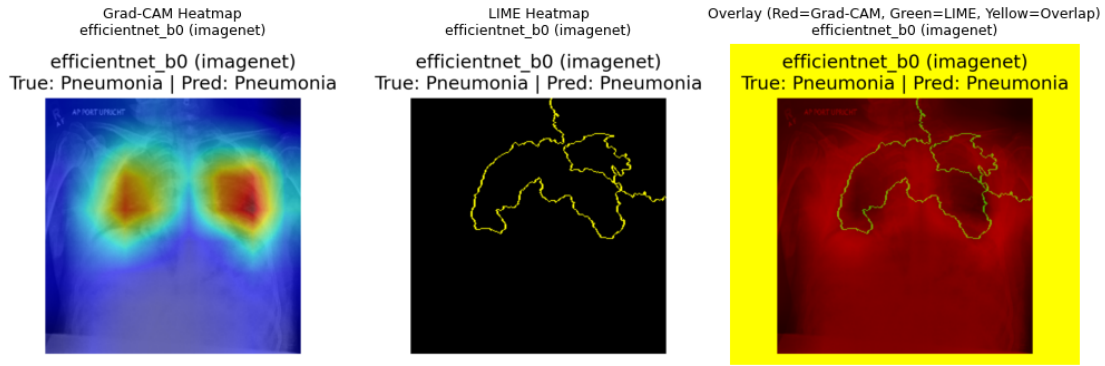
# 11. Print confirmation so we can track which combinations were
→successfully created.
print(f'[Saved] {out_path.relative_to(PROJECT_ROOT)}\n')
```



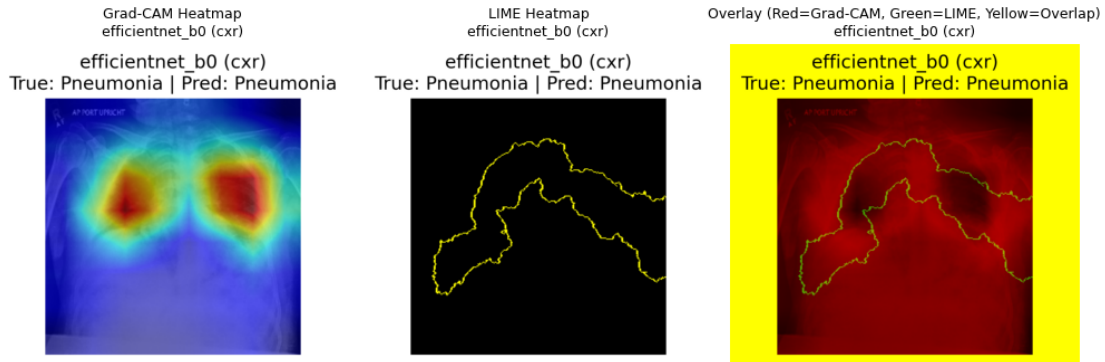
[Saved] reports/figures/explainability/mobilenet_v2_imagenet_Pneumonia_overlay_t
riptych.png



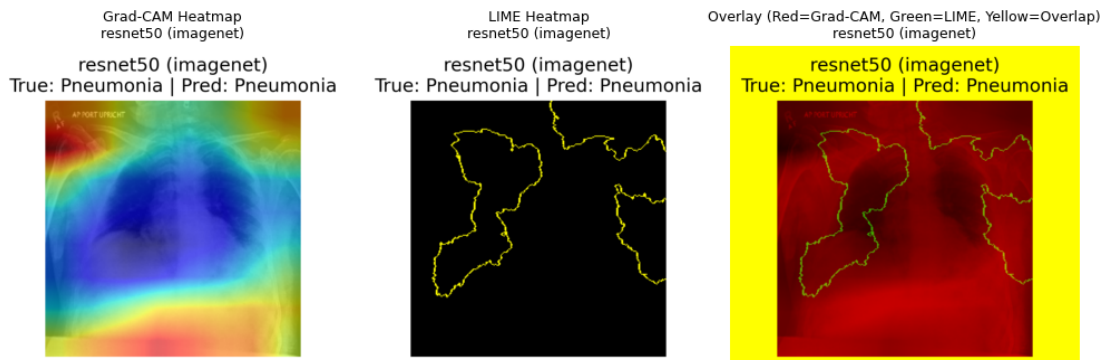
[Saved]
reports/figures/explainability/mobilenet_v2_cxr_Pneumonia_overlay_triptych.png



[Saved] reports/figures/explainability/efficientnet_b0_imagenet_Pneumonia_overlay_triptych.png

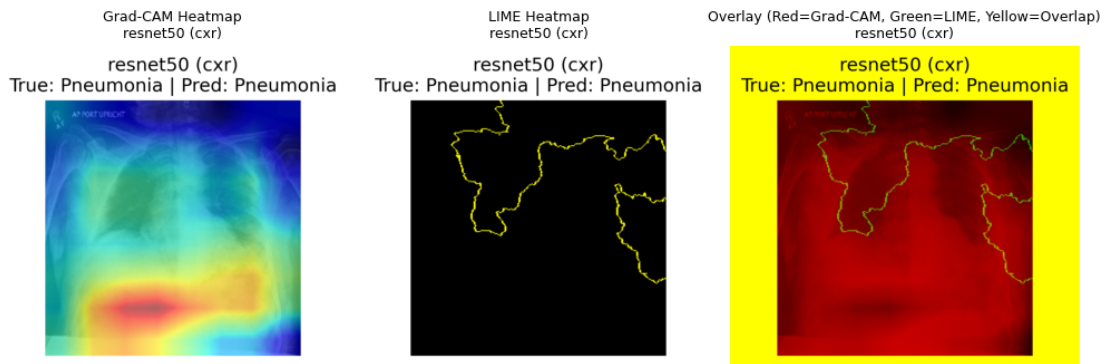


[Saved] reports/figures/explainability/efficientnet_b0_cxr_Pneumonia_overlay_triptych.png



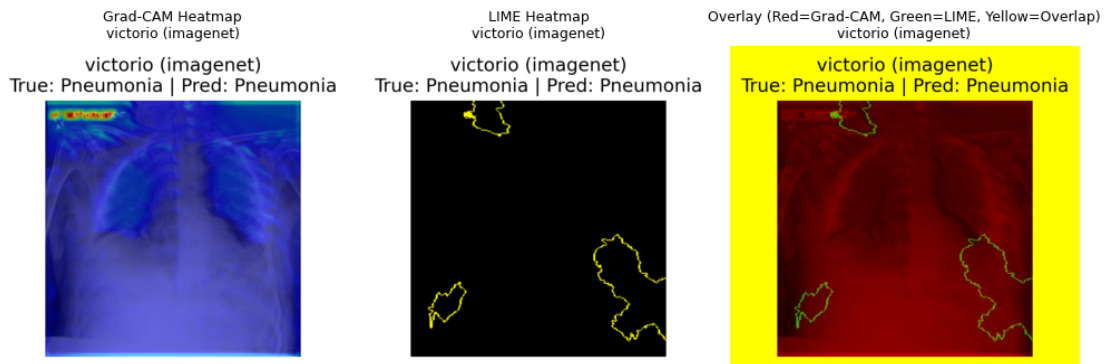
[Saved]

reports/figures/explainability/resnet50_imagenet_Pneumonia_overlay_triptych.png



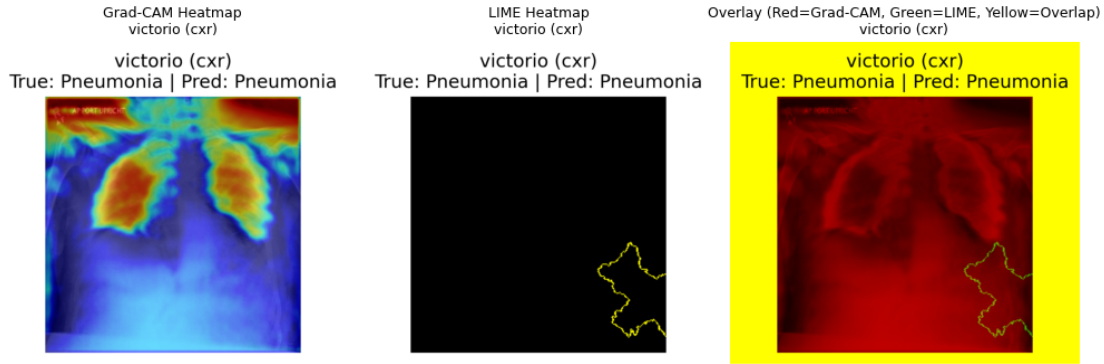
[Saved]

reports/figures/explainability/resnet50_cxr_Pneumonia_overlay_triptych.png



[Saved]

reports/figures/explainability/victorio_imagenet_Pneumonia_overlay_triptych.png



[Saved]

reports/figures/explainability/victorio_cxr_Pneumonia_overlay_trptych.png

0.12 12. Outputs and Integration.

Consolidate all explainability artifacts produced in the project, linking them to the forthcoming deployment and documentation phases. ##### Rationale: These outputs provide quantitative and visual evidence of model interpretability, ensuring reproducibility and transparency before containerisation and deployment. ##### Main Outputs: - experiments/tables/agreement_index.csv: Agreement Index (IoU) values for Grad-CAM $\times$ LIME across models and domains. - reports/figures/explainability/agreement_index_summary.png: Mean $\pm$ SD summary figure for explainability consistency. - reports/figures/explainability/*_Pneumonia_overlay_trptych.png: Eight diagnostic triptychs (Grad-CAM | LIME | Overlay) verifying spatial alignment.